COMP102: DATA STRUCTURES WITH PYTHON

Stacks & Queues

Jalal Maaouni
March 27, 2026

Plan

1. Learning Outcomes
2. What are abstract data types?
3. Stacks
4. Queues
5. Python Choices and Complexity
6. Encapsulating restricted access
7. Pitfalls & Takeaways

Learning Outcomes

Learning outcomes

By the end of this lecture, you should be able to:

Learning outcomes

By the end of this lecture, you should be able to:

- explain what an **abstract data type (ADT)** is;

Learning outcomes

By the end of this lecture, you should be able to:

- explain what an **abstract data type (ADT)** is;
- distinguish **stack** and **queue** by their access discipline;

Learning outcomes

By the end of this lecture, you should be able to:

- explain what an **abstract data type (ADT)** is;
- distinguish **stack** and **queue** by their access discipline;
- describe and use the core operations of each structure;

Learning outcomes

By the end of this lecture, you should be able to:

- explain what an **abstract data type (ADT)** is;
- distinguish **stack** and **queue** by their access discipline;
- describe and use the core operations of each structure;
- trace the state of a stack or queue after a sequence of operations;

Learning outcomes

By the end of this lecture, you should be able to:

- explain what an **abstract data type (ADT)** is;
- distinguish **stack** and **queue** by their access discipline;
- describe and use the core operations of each structure;
- trace the state of a stack or queue after a sequence of operations;
- choose an appropriate Python implementation and justify it;

Learning outcomes

By the end of this lecture, you should be able to:

- explain what an **abstract data type (ADT)** is;
- distinguish **stack** and **queue** by their access discipline;
- describe and use the core operations of each structure;
- trace the state of a stack or queue after a sequence of operations;
- choose an appropriate Python implementation and justify it;
- reason about the cost of the main operations;

Learning outcomes

By the end of this lecture, you should be able to:

- explain what an **abstract data type (ADT)** is;
- distinguish **stack** and **queue** by their access discipline;
- describe and use the core operations of each structure;
- trace the state of a stack or queue after a sequence of operations;
- choose an appropriate Python implementation and justify it;
- reason about the cost of the main operations;
- recognize common algorithmic and real-world use cases.

Learning outcomes

By the end of this lecture, you should be able to:

- explain what an **abstract data type (ADT)** is;
- distinguish **stack** and **queue** by their access discipline;
- describe and use the core operations of each structure;
- trace the state of a stack or queue after a sequence of operations;
- choose an appropriate Python implementation and justify it;
- reason about the cost of the main operations;
- recognize common algorithmic and real-world use cases.

The point of the lecture is not to memorize method names. The point is to understand **restricted access** as a design choice.

What are abstract data types?

Abstract data types (ADTs): definition

Abstract data types (ADTs): definition

What is an ADT?

An **abstract data type** specifies:

Abstract data types (ADTs): definition

What is an ADT?

An **abstract data type** specifies:

- a **set of values** (the *state*); and

Abstract data types (ADTs): definition

What is an ADT?

An **abstract data type** specifies:

- a **set of values** (the *state*); and
- a **set of operations** (the *interface*); and

Abstract data types (ADTs): definition

What is an ADT?

An **abstract data type** specifies:

- a **set of values** (the *state*); and
- a **set of operations** (the *interface*); and
- the **behavioral rules** those operations must follow.

Abstract data types (ADTs): definition

What is an ADT?

An **abstract data type** specifies:

- a **set of values** (the *state*); and
- a **set of operations** (the *interface*); and
- the **behavioral rules** those operations must follow.

Key idea

An ADT describes **what** behavior we want, *not* **how** it is represented in memory.

ADT vs data structure (DS)

ADT vs data structure (DS)

ADT (specification)

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)
- independent of language and memory layout

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)
- independent of language and memory layout
- can have multiple valid implementations

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)
- independent of language and memory layout
- can have multiple valid implementations

Data structure (implementation)

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)
- independent of language and memory layout
- can have multiple valid implementations

Data structure (implementation)

- concrete representation in memory

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)
- independent of language and memory layout
- can have multiple valid implementations

Data structure (implementation)

- concrete representation in memory
- algorithms for the operations

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)
- independent of language and memory layout
- can have multiple valid implementations

Data structure (implementation)

- concrete representation in memory
- algorithms for the operations
- performance characteristics (time/space)

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)
- independent of language and memory layout
- can have multiple valid implementations

Data structure (implementation)

- concrete representation in memory
- algorithms for the operations
- performance characteristics (time/space)
- language/library-specific choices

ADT vs data structure (DS)

ADT (specification)

- interface + meaning of operations
- invariants / rules (what must always be true)
- independent of language and memory layout
- can have multiple valid implementations

Data structure (implementation)

- concrete representation in memory
- algorithms for the operations
- performance characteristics (time/space)
- language/library-specific choices

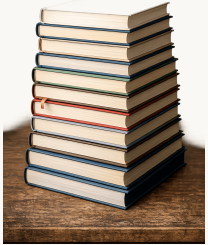
A good way to say it: **an ADT is a contract**; **a data structure is a way to satisfy that contract**.

Stacks

Intuition

Cds on a spindle.

Pile of dishes.



Pile of books.

Stack: last in, first out (LIFO)

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item
- `peek()` or `top()`: inspect the top item

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item
- `peek()` or `top()`: inspect the top item
- `is_empty()`: check whether the stack has no elements

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item
- `peek()` or `top()`: inspect the top item
- `is_empty()`: check whether the stack has no elements
- `length()`: returns the number of elements in the stack

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item
- `peek()` or `top()`: inspect the top item
- `is_empty()`: check whether the stack has no elements
- `length()`: returns the number of elements in the stack

Mental model

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item
- `peek()` or `top()`: inspect the top item
- `is_empty()`: check whether the stack has no elements
- `length()`: returns the number of elements in the stack

Mental model

Typical examples

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item
- `peek()` or `top()`: inspect the top item
- `is_empty()`: check whether the stack has no elements
- `length()`: returns the number of elements in the stack

Mental model

Typical examples

- plates

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item
- `peek()` or `top()`: inspect the top item
- `is_empty()`: check whether the stack has no elements
- `length()`: returns the number of elements in the stack

Mental model

Typical examples

- plates
- undo history

Stack: last in, first out (LIFO)

- A stack removes the **most recently inserted** item first.
- Only one end matters: the **top**.
- This is the **LIFO** discipline.

Core operations

- `push(x)`: add `x` on top
- `pop()`: remove and return the top item
- `peek()` or `top()`: inspect the top item
- `is_empty()`: check whether the stack has no elements
- `length()`: returns the number of elements in the stack

Mental model

Typical examples

- plates
- undo history
- function calls

Tracing a stack step by step

Start with an empty stack and apply the following operations:

`push(4), push(9), push(2), pop(), push(7), peek()`

Step	**Stack content (bottom → top)**	**Returned value**
start	[]	—
`push(4)`		

Tracing a stack step by step

Start with an empty stack and apply the following operations:

`push(4), push(9), push(2), pop(), push(7), peek()`

Step	**Stack content (bottom → top)**	**Returned value**
start	[]	—
`push(4)`	[4]	—
`push(9)`		

Tracing a stack step by step

Start with an empty stack and apply the following operations:

`push(4), push(9), push(2), pop(), push(7), peek()`

Step	Stack content (bottom → top)	Returned value
start	`[]`	—
`push(4)`	`[4]`	—
`push(9)`	`[4, 9]`	—
`push(2)`		

Tracing a stack step by step

Start with an empty stack and apply the following operations:

`push(4), push(9), push(2), pop(), push(7), peek()`

Step	Stack content (bottom → top)	Returned value
start	`[]`	—
`push(4)`	`[4]`	—
`push(9)`	`[4, 9]`	—
`push(2)`	`[4, 9, 2]`	—
`pop()`		

Tracing a stack step by step

Start with an empty stack and apply the following operations:

`push(4), push(9), push(2), pop(), push(7), peek()`

Step	Stack content (bottom → top)	Returned value
start	`[]`	—
`push(4)`	`[4]`	—
`push(9)`	`[4, 9]`	—
`push(2)`	`[4, 9, 2]`	—
`pop()`	`[4, 9]`	2
`push(7)`		

Tracing a stack step by step

Start with an empty stack and apply the following operations:

`push(4), push(9), push(2), pop(), push(7), peek()`

Step	Stack content (bottom → top)	Returned value
start	`[]`	—
`push(4)`	`[4]`	—
`push(9)`	`[4, 9]`	—
`push(2)`	`[4, 9, 2]`	—
`pop()`	`[4, 9]`	2
`push(7)`	`[4, 9, 7]`	—
`peek()`		

Tracing a stack step by step

Start with an empty stack and apply the following operations:

`push(4), push(9), push(2), pop(), push(7), peek()`

Step	Stack content (bottom → top)	Returned value
start	`[]`	—
`push(4)`	`[4]`	—
`push(9)`	`[4, 9]`	—
`push(2)`	`[4, 9, 2]`	—
`pop()`	`[4, 9]`	2
`push(7)`	`[4, 9, 7]`	—
`peek()`	`[4, 9, 7]`	7

Tracing a stack step by step

Start with an empty stack and apply the following operations:

`push(4), push(9), push(2), pop(), push(7), peek()`

Step	Stack content (bottom → top)	Returned value
start	`[]`	—
`push(4)`	`[4]`	—
`push(9)`	`[4, 9]`	—
`push(2)`	`[4, 9, 2]`	—
`pop()`	`[4, 9]`	2
`push(7)`	`[4, 9, 7]`	—
`peek()`	`[4, 9, 7]`	7

For a stack, the next element to leave is always the current **top**.

Implementing a stack in Python

Implementing a stack in Python

A Python `list` works well as a stack because the end of the list is efficient to use.

Implementing a stack in Python

A Python `list` works well as a stack because the end of the list is efficient to use.

Direct use of `list`

```python
stack = []

stack.append(4)   # push(4)
stack.append(9)   # push(9)
stack.append(2)   # push(2)

x = stack.pop()   # x = 2

top = stack[-1]   # peek -> 9
empty = len(stack) == 0
```

Implementing a stack in Python

A Python `list` works well as a stack because the end of the list is efficient to use.

Direct use of `list`

```python
stack = []

stack.append(4)   # push(4)
stack.append(9)   # push(9)
stack.append(2)   # push(2)

x = stack.pop()   # x = 2

top = stack[-1]   # peek -> 9
empty = len(stack) == 0
```

Example 3.4. Why this is a good fit

Implementing a stack in Python

A Python `list` works well as a stack because the end of the list is efficient to use.

Direct use of `list`

```python
stack = []

stack.append(4)   # push(4)
stack.append(9)   # push(9)
stack.append(2)   # push(2)

x = stack.pop()   # x = 2

top = stack[-1]   # peek -> 9
empty = len(stack) == 0
```

Example 3.5. Why this is a good fit

`append()` and `pop()` at the end of a list are efficient operations, which matches the one-end discipline of a stack.

Where stacks appear

Where stacks appear

Real systems

Where stacks appear

Real systems

- browser back / undo history

Where stacks appear

Real systems

- browser back / undo history
- function call stack

Where stacks appear

Real systems

- browser back / undo history
- function call stack
- expression evaluation

Where stacks appear

Real systems

- browser back / undo history
- function call stack
- expression evaluation

Programming problems

Where stacks appear

Real systems

- browser back / undo history
- function call stack
- expression evaluation

Programming problems

- balanced parentheses

Where stacks appear

Real systems

- browser back / undo history
- function call stack
- expression evaluation

Programming problems

- balanced parentheses
- reverse Polish notation

Where stacks appear

Real systems

- browser back / undo history
- function call stack
- expression evaluation

Programming problems

- balanced parentheses
- reverse Polish notation
- depth-first search (DFS)

Where stacks appear

Real systems

- browser back / undo history
- function call stack
- expression evaluation

Programming problems

- balanced parentheses
- reverse Polish notation
- depth-first search (DFS)

A stack is often the right tool when the problem depends on the **most recent unfinished context**.

Queues

Intuition

Waiting in line at service counter.

Industrial conveyor belt and processing station.

Queue: first in, first out (FIFO)

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item
- `front()` or `peek()`: inspect the front item

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item
- `front()` or `peek()`: inspect the front item
- `is_empty()`: check whether the queue has no elements

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item
- `front()` or `peek()`: inspect the front item
- `is_empty()`: check whether the queue has no elements
- `length()`: returns the number of elements in the stack

Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Mental model

Front Rear

4	9	2

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item
- `front()` or `peek()`: inspect the front item
- `is_empty()`: check whether the queue has no elements
- `length()`: returns the number of elements in the stack

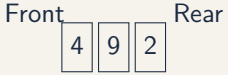
Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item
- `front()` or `peek()`: inspect the front item
- `is_empty()`: check whether the queue has no elements
- `length()`: returns the number of elements in the stack

Mental model



Typical examples

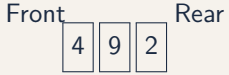
Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item
- `front()` or `peek()`: inspect the front item
- `is_empty()`: check whether the queue has no elements
- `length()`: returns the number of elements in the stack

Mental model



Typical examples

- waiting line

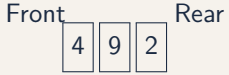
Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item
- `front()` or `peek()`: inspect the front item
- `is_empty()`: check whether the queue has no elements
- `length()`: returns the number of elements in the stack

Mental model



Typical examples

- waiting line
- printer jobs

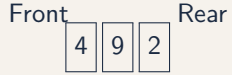
Queue: first in, first out (FIFO)

- A queue removes the **oldest waiting** item first.
- One end is used to add elements; the other is used to remove them.
- This is the **FIFO** discipline.

Core operations

- `enqueue(x)`: add `x` at the rear
- `dequeue()`: remove and return the front item
- `front()` or `peek()`: inspect the front item
- `is_empty()`: check whether the queue has no elements
- `length()`: returns the number of elements in the stack

Mental model



Typical examples

- waiting line
- printer jobs
- Task scheduling

Tracing a queue step by step

Start with an empty queue and apply the following operations:

`enqueue(4), enqueue(9), enqueue(2), dequeue(), enqueue(7), front()`

Step	**Queue content (front → rear)**	**Returned value**
start	[]	—
`enqueue(4)`		

Tracing a queue step by step

Start with an empty queue and apply the following operations:

`enqueue(4), enqueue(9), enqueue(2), dequeue(), enqueue(7), front()`

Step	**Queue content (front → rear)**	**Returned value**
start	`[]`	—
`enqueue(4)`	`[4]`	—
`enqueue(9)`		

Tracing a queue step by step

Start with an empty queue and apply the following operations:

`enqueue(4), enqueue(9), enqueue(2), dequeue(), enqueue(7), front()`

Step	**Queue content (front → rear)**	**Returned value**
start	`[]`	—
`enqueue(4)`	`[4]`	—
`enqueue(9)`	`[4, 9]`	—
`enqueue(2)`		

Tracing a queue step by step

Start with an empty queue and apply the following operations:

`enqueue(4), enqueue(9), enqueue(2), dequeue(), enqueue(7), front()`

Step	**Queue content (front → rear)**	**Returned value**
start	`[]`	—
`enqueue(4)`	`[4]`	—
`enqueue(9)`	`[4, 9]`	—
`enqueue(2)`	`[4, 9, 2]`	—
`dequeue()`		

Tracing a queue step by step

Start with an empty queue and apply the following operations:

`enqueue(4), enqueue(9), enqueue(2), dequeue(), enqueue(7), front()`

Step	**Queue content (front → rear)**	**Returned value**
start	`[]`	—
`enqueue(4)`	`[4]`	—
`enqueue(9)`	`[4, 9]`	—
`enqueue(2)`	`[4, 9, 2]`	—
`dequeue()`	`[9, 2]`	4
`enqueue(7)`		

Tracing a queue step by step

Start with an empty queue and apply the following operations:

`enqueue(4), enqueue(9), enqueue(2), dequeue(), enqueue(7), front()`

Step	**Queue content (front → rear)**	**Returned value**
start	`[]`	—
`enqueue(4)`	`[4]`	—
`enqueue(9)`	`[4, 9]`	—
`enqueue(2)`	`[4, 9, 2]`	—
`dequeue()`	`[9, 2]`	4
`enqueue(7)`	`[9, 2, 7]`	—
`front()`		

Tracing a queue step by step

Start with an empty queue and apply the following operations:

`enqueue(4), enqueue(9), enqueue(2), dequeue(), enqueue(7), front()`

Step	Queue content (front → rear)	Returned value
start	`[]`	—
`enqueue(4)`	`[4]`	—
`enqueue(9)`	`[4, 9]`	—
`enqueue(2)`	`[4, 9, 2]`	—
`dequeue()`	`[9, 2]`	4
`enqueue(7)`	`[9, 2, 7]`	—
`front()`	`[9, 2, 7]`	9

For a queue, the next element to leave is always the one that has been waiting the longest.

Tracing a queue step by step

Start with an empty queue and apply the following operations:

`enqueue(4), enqueue(9), enqueue(2), dequeue(), enqueue(7), front()`

Step	Queue content (front → rear)	Returned value
start	`[]`	—
`enqueue(4)`	`[4]`	—
`enqueue(9)`	`[4, 9]`	—
`enqueue(2)`	`[4, 9, 2]`	—
`dequeue()`	`[9, 2]`	4
`enqueue(7)`	`[9, 2, 7]`	—
`front()`	`[9, 2, 7]`	9

For a queue, the next element to leave is always the one that has been waiting the longest.

Implementing a queue in Python

For queues, the standard Python tool is usually `collections.deque`.

Implementing a queue in Python

For queues, the standard Python tool is usually `collections.deque`.

Implementing a queue in Python

Recommended approach

```python
from collections import deque

queue = deque()

queue.append(4)       # enqueue(4)
queue.append(9)       # enqueue(9)
queue.append(2)       # enqueue(2)

x = queue.popleft()   # x = 4

front = queue[0]      # peek/front -> 9
empty = len(queue) == 0
```

Implementing a queue in Python

Recommended approach

```python
from collections import deque

queue = deque()

queue.append(4)       # enqueue(4)
queue.append(9)       # enqueue(9)
queue.append(2)       # enqueue(2)

x = queue.popleft()   # x = 4

front = queue[0]      # peek/front -> 9
empty = len(queue) == 0
```

Example 4.4. Why deque?

Implementing a queue in Python

Recommended approach

```python
from collections import deque

queue = deque()

queue.append(4)       # enqueue(4)
queue.append(9)       # enqueue(9)
queue.append(2)       # enqueue(2)

x = queue.popleft()   # x = 4

front = queue[0]      # peek/front -> 9
empty = len(queue) == 0
```

Example 4.5. Why deque?

A queue needs efficient operations at opposite ends. That is exactly what deque is designed for.

Why `list.pop(0)` is a bad habit for queues

Why `list.pop(0)` is a bad habit for queues

A Python list can *simulate* a queue, but it is usually not the right choice.

Why `list.pop(0)` is a bad habit for queues

A Python list can *simulate* a queue, but it is usually not the right choice.

Naive version

```
queue = []
queue.append(4)
queue.append(9)
queue.append(2)

x = queue.pop(0)    # works, but not a good implementation choice
```

Why `list.pop(0)` is a bad habit for queues

A Python list can *simulate* a queue, but it is usually not the right choice.

Naive version

```
queue = []
queue.append(4)
queue.append(9)
queue.append(2)

x = queue.pop(0)    # works, but not a good implementation choice
```

- The code is **behaviorally correct**: it does remove the oldest item.

Why `list.pop(0)` is a bad habit for queues

A Python list can *simulate* a queue, but it is usually not the right choice.

Naive version

```
queue = []
queue.append(4)
queue.append(9)
queue.append(2)

x = queue.pop(0)    # works, but not a good implementation choice
```

- The code is **behaviorally correct**: it does remove the oldest item.
- But removing index 0 shifts the remaining elements.

Why `list.pop(0)` is a bad habit for queues

A Python list can *simulate* a queue, but it is usually not the right choice.

Naive version

```
queue = []
queue.append(4)
queue.append(9)
queue.append(2)

x = queue.pop(0)    # works, but not a good implementation choice
```

- The code is **behaviorally correct**: it does remove the oldest item.
- But removing index 0 shifts the remaining elements.
- So a solution can be correct and still be the wrong implementation choice.

Python Choices and Complexity

Choosing the right implementation

ADT	Operation	Good Python tool	Intuition	Complexity
Stack	push	`list.append()`	efficient at the end	O(1) amortized
Stack	pop	`list.pop()`	efficient at the end	O(1) amortized
Queue	enqueue	`deque.append()`	efficient at the rear	O(1) amortized
Queue	dequeue	`deque.popleft()`	efficient at the front	O(1) amortized
Queue (naive)	dequeue	`list.pop(0)`	shifting cost	O(n)

Choosing the right implementation

ADT	Operation	Good Python tool	Intuition	Complexity
Stack	push	`list.append()`	efficient at the end	O(1) amortized
Stack	pop	`list.pop()`	efficient at the end	O(1) amortized
Queue	enqueue	`deque.append()`	efficient at the rear	O(1) amortized
Queue	dequeue	`deque.popleft()`	efficient at the front	O(1) amortized
Queue (naive)	dequeue	`list.pop(0)`	shifting cost	O(n)

The ADT tells us **what** behavior we want. Complexity helps us choose **how** to implement it.

Stack vs queue: side-by-side recap

Feature	**Stack**	**Queue**
Discipline	LIFO	FIFO
Insert operation	`push`	`enqueue`
Remove operation	`pop`	`dequeue`
Where items leave	top	front
Typical Python tool	`list`	`deque`
Common uses	undo, DFS, parsing	scheduling, BFS, buffering

Encapsulating restricted access

Why wrap stacks and queues in classes?

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

What can go wrong?

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

What can go wrong?

- a stack user can do `stack.insert(0, x)`

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

What can go wrong?

- a stack user can do `stack.insert(0, x)`
- a queue user can do `queue.pop()`

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

What can go wrong?

- a stack user can do `stack.insert(0, x)`
- a queue user can do `queue.pop()`
- code can depend on details of the container

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

What can go wrong?

- a stack user can do `stack.insert(0, x)`
- a queue user can do `queue.pop()`
- code can depend on details of the container

OOP idea

We hide the representation and expose only:

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

What can go wrong?

- a stack user can do `stack.insert(0, x)`
- a queue user can do `queue.pop()`
- code can depend on details of the container

OOP idea

We hide the representation and expose only:

- `push`, `pop`, `peek`

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

What can go wrong?

- a stack user can do `stack.insert(0, x)`
- a queue user can do `queue.pop()`
- code can depend on details of the container

OOP idea

We hide the representation and expose only:

- `push`, `pop`, `peek`
- `enqueue`, `dequeue`, `front`

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Why wrap stacks and queues in classes?

Problem with the direct approach

When we write `stack = []` or `queue = deque()`, the underlying container is fully exposed. Nothing stops us from doing operations that violate the intended discipline.

What can go wrong?

- a stack user can do `stack.insert(0, x)`
- a queue user can do `queue.pop()`
- code can depend on details of the container

OOP idea

We hide the representation and expose only:

- `push`, `pop`, `peek`
- `enqueue`, `dequeue`, `front`
- `is_empty`

A class helps enforce restricted access: users interact with the *ADT operations*, not with the raw container.

Template: MyStack

Class skeleton

This template defines the interface of a stack without implementing it yet.

```python
class MyStack:
    def __init__(self):
        pass

    def push(self, value):
        pass

    def pop(self):
        pass

    def peek(self):
        pass

    def is_empty(self):
        pass
```

Template: `MyQueue`

Class skeleton

This template defines the interface of a queue without implementing it yet.

```python
class MyQueue:
    def __init__(self):
        pass

    def enqueue(self, value):
        pass

    def dequeue(self):
        pass

    def front(self):
        pass

    def is_empty(self):
        pass
```

Pitfalls & Takeaways

Common mistakes

Common mistakes

- Confusing `peek()` with `pop()`.

Common mistakes

- Confusing `peek()` with `pop()`.
- Removing from the wrong side.

Common mistakes

- Confusing `peek()` with `pop()`.
- Removing from the wrong side.
- Using `list.pop(0)` for queues without thinking about cost.

Common mistakes

- Confusing `peek()` with `pop()`.
- Removing from the wrong side.
- Using `list.pop(0)` for queues without thinking about cost.
- Forgetting to handle the empty case.

Common mistakes

- Confusing `peek()` with `pop()`.
- Removing from the wrong side.
- Using `list.pop(0)` for queues without thinking about cost.
- Forgetting to handle the empty case.
- Treating *behavior* and *implementation* as if they were the same idea.

Common mistakes

- Confusing `peek()` with `pop()`.
- Removing from the wrong side.
- Using `list.pop(0)` for queues without thinking about cost.
- Forgetting to handle the empty case.
- Treating *behavior* and *implementation* as if they were the same idea.

A data structure is not just a place to store values. It is a promise about which operations are natural, efficient, and meaningful.

Takeaways

These structures are simple, but they are foundational. They reappear inside parsers, search algorithms, schedulers, interpreters, and many programming problems.

Takeaways

- A **stack** is a restricted-access structure with **LIFO** behavior.

These structures are simple, but they are foundational. They reappear inside parsers, search algorithms, schedulers, interpreters, and many programming problems.

Takeaways

- A **stack** is a restricted-access structure with **LIFO** behavior.
- A **queue** is a restricted-access structure with **FIFO** behavior.

These structures are simple, but they are foundational. They reappear inside parsers, search algorithms, schedulers, interpreters, and many programming problems.

Takeaways

- A **stack** is a restricted-access structure with **LIFO** behavior.
- A **queue** is a restricted-access structure with **FIFO** behavior.
- These are **ADTs**: they describe behavior before implementation.

These structures are simple, but they are foundational. They reappear inside parsers, search algorithms, schedulers, interpreters, and many programming problems.

Takeaways

- A **stack** is a restricted-access structure with **LIFO** behavior.
- A **queue** is a restricted-access structure with **FIFO** behavior.
- These are **ADTs**: they describe behavior before implementation.
- In Python, `list` is a good fit for stacks; `deque` is a good fit for queues.

These structures are simple, but they are foundational. They reappear inside parsers, search algorithms, schedulers, interpreters, and many programming problems.

Takeaways

- A **stack** is a restricted-access structure with **LIFO** behavior.
- A **queue** is a restricted-access structure with **FIFO** behavior.
- These are **ADTs**: they describe behavior before implementation.
- In Python, `list` is a good fit for stacks; `deque` is a good fit for queues.
- Correct behavior is not enough: implementation choice affects performance.

These structures are simple, but they are foundational. They reappear inside parsers, search algorithms, schedulers, interpreters, and many programming problems.

Takeaways

- A **stack** is a restricted-access structure with **LIFO** behavior.
- A **queue** is a restricted-access structure with **FIFO** behavior.
- These are **ADTs**: they describe behavior before implementation.
- In Python, `list` is a good fit for stacks; `deque` is a good fit for queues.
- Correct behavior is not enough: implementation choice affects performance.

These structures are simple, but they are foundational. They reappear inside parsers, search algorithms, schedulers, interpreters, and many programming problems.